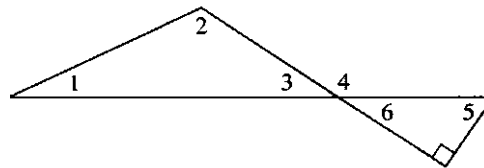


24. Suponha que você usou os passos do Exemplo 9 para tentar mostrar que $\sqrt{4}$ não é um número racional. Em qual passo a prova não seria válida?
25. Prove que $\sqrt{3}$ não é um número racional.
26. Prove que $\sqrt{5}$ não é um número racional.
27. Prove que $\sqrt[3]{2}$ não é um número racional.
- ★28. Prove ou apresente um contra-exemplo: O produto de quaisquer três inteiros consecutivos é par.
29. Prove ou apresente um contra-exemplo: A soma de quaisquer três inteiros consecutivos é par.
30. Prove ou apresente um contra-exemplo: O produto de um inteiro pelo seu quadrado é par.
- ★31. Prove ou apresente um contra-exemplo: A soma de um inteiro com o seu cubo é par.
32. Prove ou apresente um contra-exemplo: Para um inteiro positivo x , $x + \frac{1}{x} \geq 2$.
33. Prove ou apresente um contra-exemplo: Para todo número primo n , $n + 4$ é primo.
34. Prove ou apresente um contra-exemplo: O produto de dois números irracionais é irracional.
- ★35. Prove ou apresente um contra-exemplo: A soma de dois números racionais é racional.

Para os exercícios 36 a 38, use a figura abaixo e os seguintes fatos da geometria:

- A soma dos ângulos internos de um triângulo é 180° .
- Ângulos opostos pelo vértice (ângulos opostos que são formados quando duas linhas se interceptam) têm a mesma medida.
- Um ângulo raso mede 180° .
- Um ângulo reto mede 90° .



36. Prove que a medida do ângulo 4 é a soma das medidas dos ângulos 1 e 2.
- ★37. Prove que a medida do ângulo 5 mais a medida do ângulo 3 é 90° .
38. Se o ângulo 1 e o ângulo 5 têm a mesma medida, então o ângulo 2 é um ângulo reto.

Seção 2.2 Indução

O Método

Existe uma última técnica de demonstração que se aplica a determinadas situações. Para ilustrar o uso desta técnica, imagine que você está subindo em uma escada sem fim. Como você pode saber se será capaz de alcançar um degrau arbitrariamente alto? Suponha que você faça as seguintes afirmações sobre as suas habilidades de subir escadas:

1. Você pode alcançar o primeiro degrau.
2. Se você alcançar um degrau, você pode sempre passar ao degrau seguinte. (Note que esta asserção é uma implicação.)

Tanto a sentença 1 como a implicação na sentença 2 são verdadeiras; então, pela sentença 1 você pode chegar ao primeiro degrau e pela sentença 2 você pode chegar ao segundo; novamente pela 2 você pode chegar ao terceiro; pela sentença 2 novamente você pode chegar ao quarto degrau, e assim sucessivamente. Você pode, então, subir tão alto quanto você queira. Neste caso, ambas as asserções são necessárias. Se apenas a sentença 1 é verdadeira, você não tem garantias de que poderá ir além do primeiro degrau, e se apenas a segunda sentença é verdadeira, você poderá não chegar ao primeiro degrau a fim de iniciar o processo de subida da escada. Vamos assumir que os degraus da escada são numerados com os números inteiros positivos — 1, 2, 3, e assim por diante. Agora vamos considerar uma propriedade específica que um número pode ter. Ao invés de "alcançarmos um degrau arbitrário" podemos mencionar que um inteiro positivo arbitrário tem essa propriedade. Usaremos a notação simplificada $P(n)$ para denotar que o inteiro positivo n tem a propriedade P . Como podemos usar a técnica de subir escadas para provar que para todos inteiros positivos n nós temos $P(n)$? As duas afirmações de que precisamos para a demonstração são:

- | | |
|--|---|
| 1. $P(1)$ | (1 tem a propriedade P .) |
| 2. Para qualquer inteiro positivo k ,
$P(k) \rightarrow P(k+1)$ | (Se algum número tem a propriedade P , então o número seguinte também a tem.) |

Se pudermos demonstrar as sentenças 1 e 2, então $P(n)$ vale para qualquer inteiro positivo n , da mesma maneira que nós podemos subir até um degrau arbitrário na escada.

O fundamento deste tipo de argumentação é chamado **princípio de indução matemática**, que pode ser enunciado como

- | | |
|--|--|
| 1. $P(1)$ verdadeira | } $\rightarrow P(n)$ verdadeira para todos os n inteiros positivos |
| 2. $(\forall k) [P(k) \text{ verdadeira} \rightarrow P(k+1) \text{ verdadeira}]$ | |

O princípio da indução matemática é uma implicação. A tese desta implicação é uma sentença da forma " $P(n)$ é verdadeira para todos os n inteiros positivos". Portanto, sempre que desejamos demonstrar que alguma propriedade é válida para todo inteiro positivo n , uma tentativa é o uso da indução matemática como técnica de demonstração.

Para averiguarmos que a tese desta implicação é verdadeira, mostramos que as duas hipóteses, ou seja, as afirmações 1 e 2 são verdadeiras. Para demonstrar a afirmação 1, precisamos apenas mostrar que a propriedade P vale para o número 1, o que é normalmente uma tarefa trivial. Para demonstrar a afirmação 2, uma implicação que deve valer para todo k , assumimos que $P(k)$ é verdadeira para um inteiro arbitrário k , e baseados nesta hipótese mostramos que $P(k+1)$ é verdadeira. Você deve convencer a si próprio que assumir a propriedade P como válida para o número k não é a mesma coisa que assumir o que desejamos demonstrar (uma freqüente fonte de confusão, quando nos deparamos pela primeira vez com este tipo de demonstração). Assumi-la como verdadeira é simplesmente o caminho para elaborar a prova de que a implicação $P(k) \rightarrow P(k+1)$ é verdadeira.

Ao desenvolver uma demonstração por indução, estabelecemos inicialmente a veracidade da sentença 1, $P(1)$, que é chamada de **base da indução** ou **passo básico**, para a demonstração indutiva. Estabelecer que a sentença $P(k) \rightarrow P(k+1)$ é verdadeira constitui o **passo indutivo**. Quando assumimos que $P(k)$ é verdadeira com o intuito de demonstrar o passo indutivo, $P(k)$ é chamado de **suposição indutiva**, ou **hipótese indutiva**.

Todos os métodos de demonstração sobre os quais comentamos neste capítulo são técnicas de raciocínio dedutivo — formas de demonstrar uma conjectura que possivelmente foi formulada por um raciocínio indutivo. A indução matemática também é uma técnica *dedutiva*, e não um método para o raciocínio indutivo (procure não ficar confuso com a terminologia utilizada aqui). Para outras técnicas de demonstração, podemos começar com as hipóteses, e encadear fatos até que mais ou menos tropeçemos em uma solução. De fato, mesmo que nossa conjectura seja ligeiramente incorreta, provavelmente acabamos por verificar a tese correta no decorrer do processo de demonstração. Na indução matemática, entretanto, devemos saber no princípio a forma exata da propriedade $P(n)$ que estamos tentando estabelecer. A indução matemática, portanto, não é uma técnica de demonstração exploratória — ela pode apenas confirmar uma conjectura correta.

Demonstrações Indutivas

Suponha que o Sr. Silva casou-se e teve dois filhos. Vamos chamar estes dois filhos de geração 1. Agora suponha que cada um desses dois filhos teve dois filhos; então na geração 2 temos quatro descendentes. Este processo continua de geração em geração. A árvore genealógica da família Silva é semelhante à Fig. 2.1. (Que é exatamente igual à Fig. 1.1b, onde buscamos os possíveis valores T-F para as n letras de afirmações.)

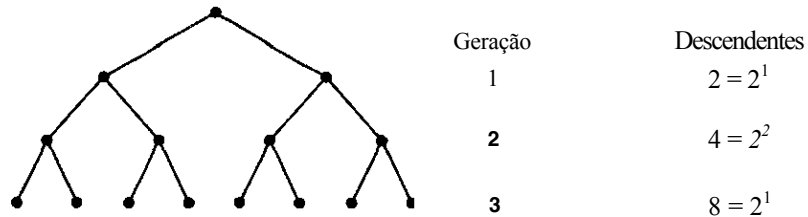


Figura 2.1

Aparentemente a geração n tem 2^n descendentes. De maneira mais formal, se fizermos $P(n)$ denotar o número de descendentes na geração n , então nossa suposição será

$$P(n) = 2^n$$

Podemos usar a indução para *demonstrar* que nosso palpite para $P(n)$ está correto.

A base da indução é estabelecer $P(1)$, que resulta a equação

$$P(1) = 2^1 = 2$$

Que é verdadeira, posto que foi informado que o Sr. Silva tinha dois filhos. Agora vamos supor que nossa premissa é verdadeira para uma geração arbitrária k , ou seja, assumimos que

$$P(k) = 2^k$$

e tentaremos mostrar que

$$P(k+1) = 2^{k+1}$$

Nesta família, cada descendente tem dois filhos; então o número de descendentes na geração $k + 1$ será o dobro do da geração índice k , ou seja, $P(k + 1) = 2 P(k)$. Pela hipótese de indução, $P(k) = 2^k$, logo

$$P(k+1) = 2 P(k) = 2(2^k) = 2^{k+1}$$

então, de fato,

$$P(k+1) = 2^{k+1}$$

Isto completa a nossa demonstração por indução. Agora que sabemos como resolver o problema simples do clã dos Silva, podemos aplicar a técnica de demonstração por indução em problemas menos óbvios.

EXEMPLO 10 Prove que a equação

$$1 + 3 + 5 + \dots + (2n-1) = n^2 \quad (1)$$

é verdadeira para qualquer inteiro positivo n . Neste caso, a propriedade $P(n)$ é que a equação (1) acima é verdadeira. O lado esquerdo desta equação é a soma de todos os inteiros ímpares de 1 até $2n - 1$. Ainda que possamos verificar que a equação é verdadeira para um particular valor de n pela substituição deste valor na equação, nós não podemos substituir todos os possíveis valores inteiros positivos. Por isso, uma demonstração por exemplos não funciona. É mais apropriada uma demonstração por indução matemática.

O passo básico é estabelecer $P(1)$, que é o valor da equação (1) quando n assume o valor 1, ou seja

$$1 = 1^2$$

Isto certamente é verdadeiro. Para a hipótese de indução, vamos assumir $P(k)$ como verdadeira para um inteiro positivo arbitrário k , que é o valor da equação (1) quando n vale k , ou seja

$$1 + 3 + 5 + \dots + (2k-1) = k^2 \quad (2)$$

(Note que $P(k)$ não é a equação $(2k-1) = k^2$ que só é verdadeira quando $k = 1$.) Usando a hipótese de indução, queremos mostrar $P(k+1)$ que é o valor da equação (1) quando n assume o valor $k+1$, ou seja:

$$1 + 3 + 5 + \dots + [2(k+1) - 1] = (k+1)^2 \quad (3)$$

(O pequeno ponto de interrogação sobre o sinal de igualdade serve para nos lembrar que é este fato que desejamos provar, a partir de um outro fato já conhecido.)

A chave para uma demonstração por indução é encontrar uma forma de relacionar o que se deseja mostrar — $P(k+1)$, equação (3) — com o que assumimos como verdadeiro — $P(k)$, equação (2). O lado esquerdo de $P(k+1)$ pode ser reescrito a fim de destacar o penúltimo termo:

$$1 + 3 + 5 + \dots + (2k-1) + [2(k+1) - 1]$$

Esta expressão contém do lado esquerdo a equação (2) como subexpressão. Como assumimos que $P(k)$ é verdadeira, podemos substituir esta expressão pelo lado direito da equação (2). Senão vejamos:

$$\begin{aligned} 1 + 3 + 5 + \dots + [2(k+1) - 1] &= 1 + 3 + 5 + \dots + (2k-1) + [2(k+1) - 1] \\ &= k^2 + [2(k+1) - 1] \\ &= k^2 + [2k + 2 - 1] \\ &= k^2 + 2k + 1 \\ &= (k+1)^2 \end{aligned}$$

Portanto

$$1 + 3 + 5 + \dots + [2(k+1) - 1] = (k+1)^2$$

o que verifica $P(k+1)$ e prova que a equação (1) é verdadeira para qualquer inteiro positivo n . •

EXEMPLO 11 Prove que

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

para qualquer $n > 1$.

Aqui, novamente, a indução é a técnica mais apropriada. $P(1)$ é a equação

$$1 + 2 = 2^{1+1} - 1 \text{ ou } 3 = 2^2 - 1$$

que é verdadeira. Consideremos agora $P(k)$

$$1 + 2 + 2^2 + \dots + 2^k = 2^{k+1} - 1$$

como a hipótese de indução, e busquemos obter $P(k+1)$:

$$1 + 2 + 2^2 + \dots + 2^{k+1} \stackrel{?}{=} 2^{k+1+1} - 1$$

Agora, se reescrevermos a soma do lado esquerdo de $P(k+1)$, obtemos uma forma de usar a hipótese de indução:

$$\begin{aligned} 1 + 2 + 2^2 + \dots + 2^{k+1} &= 1 + 2 + 2^2 + \dots + 2^k + 2^{k+1} \\ &= 2^{k+1} - 1 + 2^{k+1} \quad (\text{da hipótese de indução } P(k)) \\ &= 2(2^{k+1}) - 1 \\ &= 2^{k+1+1} - 1 \end{aligned}$$

Portanto

$$1 + 2 + 2^2 + \dots + 2^{k+1} = 2^{k+1+1} - 1$$

que verifica $P(k+1)$ e completa a demonstração.

PRÁTICA 6 Prove que para qualquer inteiro positivo n ,

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

◆

Nem todas as demonstrações por indução envolvem fórmulas com somas. Outras identidades algébricas sobre inteiros positivos podem ser demonstradas por indução, bem como suposições não-algébricas, como o número de descendentes na geração n da família Silva.

EXEMPLO 12 Demonstre que, para qualquer inteiro positivo n , $2^n > n$.

$P(1)$ é a sentença $2^1 > 1$ que é, sem dúvida, verdadeira. Suponhamos agora $P(k)$, $2^k > k$, como verdadeira, e procuremos concluir a partir daí $P(k+1)$, $2^{k+1} > k+1$. Começando pelo lado esquerdo de $P(k+1)$, temos que $2^{k+1} = 2^k \cdot 2$. Usando a hipótese de indução $2^k > k$ e multiplicando ambos os lados dessa desigualdade por 2, temos $2^k \cdot 2 > k \cdot 2$. Podemos escrever então que:

$$2^{k+1} = 2^k \cdot 2 > k \cdot 2 = k + k \geq k + 1$$

o que resulta em

$$2^{k+1} > k + 1$$

•

EXEMPLO 13 Prove que, para qualquer inteiro positivo n , o número $2^{2n} - 1$ é divisível por 3.

A base da indução é mostrar $P(1)$, ou seja, mostrar que $2^{2(1)} - 1 = 4 - 1 = 3$ é divisível por 3, o que obviamente é verdadeiro.

Assumimos que $2^{2k} - 1$ é divisível por 3, o que equivale a escrever $2^{2k} - 1 = 3m$ para algum inteiro m , ou $2^{2k} = 3m + 1$. Desejamos mostrar que $2^{2(k+1)} - 1$ é divisível por 3.

$$\begin{aligned} 2^{2(k+1)} - 1 &= 2^{2k+2} - 1 \\ &= 2^2 \cdot 2^{2k} - 1 \\ &= 2^2 \cdot (3m+1) - 1 \quad (\text{pela hipótese de indução}) \\ &= 12m + 4 - 1 \\ &= 12m + 3 \\ &= 3(4m+1) \text{ onde } 4m+1 \text{ é um inteiro.} \end{aligned}$$

Logo $2^{2(k+1)} - 1$ é divisível por 3.

•

Em alguns casos, pode ser que, para o primeiro passo do processo de indução, seja mais apropriado começar com valores 0, 2, ou 3, ao invés de 1. O mesmo princípio se aplica, qualquer que seja o degrau da escada que você alcance no primeiro passo.

EXEMPLO 14 Prove que $n^2 > 3n$ para $n \geq 4$.

Neste exemplo devemos aplicar a indução e começar com um passo inicial $P(4)$. (Se testarmos para valores de $n = 1, 2$ e 3, verificaremos que a desigualdade não se verifica.) $P(A)$ é então a desigualdade $4^2 > 3(4)$, ou $16 > 12$, que é verdadeira. A hipótese de indução é $k^2 > 3k$, para $k > 4$, e queremos mostrar que $(k+1)^2 > 3(k+1)$.

$$\begin{aligned} (k+1)^2 &= k^2 + 2k + 1 \\ &> 3k + 2k + 1 \quad (\text{pela hipótese de indução}) \\ &\geq 3k + 8 + 1 \quad (\text{já que } k \geq 4) \\ &> 3k + 3 \\ &= 3(k+1) \end{aligned}$$

PRÁTICA 7 Prove que $2^{n+1} < 3^n$ para todo $n > 1$.

•

Podemos lançar mão de uma prova por indução para aplicações que não sejam tão óbvias quanto às dos exemplos acima. Isto geralmente acontece quando a sentença que desejamos demonstrar contém uma variável que pode assumir valores inteiros arbitrários e não-negativos.

EXEMPLO 15

Uma linguagem de programação pode ser projetada com as seguintes convenções no que se refere à multiplicação. Um fator simples não requer parênteses, mas o produto "o vezes b " deve ser escrito como $(a)b$. Desejamos mostrar que qualquer produto de fatores precisa de um número par de parênteses para ser escrito. A prova é feita por indução, considerando-se como variável o número de fatores. Para um único fator usa-se 0 (zero) parêntese, um número par. Suponhamos que para qualquer produto de k fatores, usamos um número par de parênteses. Consideremos agora o produto P de $k+1$ fatores. P pode ser escrito na forma r vezes s , onde r tem k fatores e s é um fator único. Pela hipótese de indução, r tem um número par de parênteses. Então pode-se escrever r vezes s como $(r)s$, adicionando-se mais dois parênteses ao número par de parênteses em r , e resultando assim um número par de parênteses para P . •

É possível se enganar ao construir uma prova por indução. Quando demonstramos que $P(k+1)$ é verdadeira, sem nos valermos da hipótese $P(k)$, nós elaboramos uma prova direta de $P(k+1)$, onde $k+1$ é arbitrário. A prova deveria ser reescrita para explicitar que é uma prova direta de $P(n)$ para qualquer n , e não uma prova por indução.

Indução Completa

O princípio da indução que tem sido utilizado,

$$\left. \begin{array}{l} 1. P(1) \text{ verdadeira} \\ 2. (\forall k) [P(k) \text{ verdadeira} \rightarrow P(k+1) \text{ verdadeira}] \end{array} \right\} \rightarrow P(n) \text{ verdadeira para todos os } n \text{ inteiros positivos.}$$

é também conhecido como **indução fraca** em oposição ao princípio da **indução forte** ou **indução completa**:

$$\left. \begin{array}{l} 1'. P(1) \text{ verdadeira} \\ 2'. (\forall k) [P(r) \text{ verdadeira para todo } r \\ \quad 1 \leq r \leq k \rightarrow P(k+1) \text{ verdadeira}] \end{array} \right\} \rightarrow P(n) \text{ verdadeira para todos os } n \text{ inteiros positivos.}$$

Estes dois princípios de indução diferem nos itens 2 e 2'. No item 2, devemos ser capazes de provar para um inteiro positivo arbitrário k que $P(k+1)$ é verdadeira, tendo por base apenas a premissa de que $P(k)$ é verdadeira. No item 2', podemos assumir que $P(r)$ é verdadeira para todos os inteiros r entre 1 e um inteiro positivo arbitrário k , para provar que $P(k+1)$ é verdadeira. Este fato nos fornece uma boa "munição", de tal forma que podemos algumas vezes ser capazes de provar a implicação em 2', quando não podemos provar a implicação em 2.

O que nos leva a deduzir $(\forall n)P(n)$ em cada caso? Devemos notar que os dois princípios de indução, ou seja, os dois métodos de demonstração são equivalentes. Em outras palavras, se aceitarmos a indução fraca como um princípio válido, então o princípio da indução forte também será válido e vice-versa. Para provar a equivalência entre esses dois princípios de indução, apresentamos um outro princípio chamado **princípio da boa ordenação**: Toda coleção de números inteiros positivos que contenha pelo menos um elemento tem um mínimo.

Devemos então verificar que as seguintes implicações são verdadeiras:

$$\begin{array}{l} \text{indução forte} \rightarrow \text{indução fraca} \\ \text{indução fraca} \rightarrow \text{princípio da boa ordenação} \\ \text{princípio da boa ordenação} \rightarrow \text{indução forte} \end{array}$$

Como consequência, estes princípios são equivalentes, e aceitar qualquer um dos três como verdadeiro significa aceitar também os outros dois.

Para provar que o princípio da indução forte implica a indução fraca, suponhamos como premissa válida a indução forte. Desejamos então mostrar que a indução fraca é válida, ou seja, que podemos concluir $P(n)$ para todo n a partir das afirmações 1 e 2. Se a afirmação 1 é verdadeira, então a afirmação 1' também o é. Se a afirmação 2 é verdadeira, a 2' também é, porque podemos concluir $P(k+1)$ a partir de $P(r)$ para todo r entre 1 e k , apesar de usarmos apenas a condição $P(k)$. (Mais precisamente, a sentença 2' requer que provemos $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$, mas $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k)$ e, da afirmação 2, $P(k) \rightarrow P(k+1)$, logo $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$). Pela indução forte, concluímos $P(n)$ para todo n . A prova de que a indução fraca implica a boa ordenação, e que a boa ordenação implica a indução forte, é deixada como exercício na Seção 3.1.

A fim de ilustrar a diferença entre uma demonstração por indução fraca de uma demonstração por indução forte, vamos examinar um exemplo pitoresco que pode ser demonstrado das duas formas.

EXEMPLO 16 Prove que uma cerca de madeira com n estacas tem $n - 1$ seções para qualquer $n \geq 1$ (veja Fig. 2.2a).

Seja $P(n)$ a sentença de que a cerca com n estacas tem $n - 1$ seções, e provemos que $P(n)$ é verdadeira para todo $n \geq 1$.

Usaremos inicialmente a indução fraca. Pela base da indução, $P(1)$ assegura que uma cerca com apenas uma estaca tem 0 seções, o que é claramente verdadeiro (veja Fig. 2.2b). Assuma que $P(k)$ é verdadeira:

uma cerca com k estacas tem $k - 1$ seções

e tente provar $P(k+1)$:

(?) uma cerca com $k+1$ estacas tem k seções

Dada uma cerca com $k+1$ estacas, como podemos relacioná-la com uma cerca com k estacas a fim de poderemos utilizar a hipótese de indução? Podemos remover de uma cerca a última estaca e a última seção (Fig. 2.2c). A cerca remanescente tem então k estacas, e pela hipótese de indução, $k - 1$ seções. Portanto, a cerca original tinha k seções.

Demonstraremos agora o mesmo resultado utilizando a indução forte. A base da indução é a mesma de antes. Como hipótese de indução, assumimos que:

para todo r , $1 \leq r \leq k$, uma cerca com r estacas tem $r - 1$ seções

e tentaremos provar $P(k+1)$:

(?) uma cerca com $k+1$ estacas tem k seções.

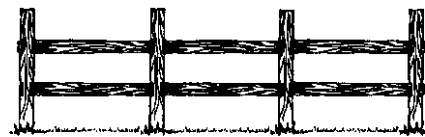
Consideremos uma cerca com $k+1$ estacas, e vamos separá-la em duas partes, removendo uma seção (Fig. 2.2d). As duas partes da cerca têm r_1 e r_2 estacas, onde $1 \leq r_1 \leq k$, $1 \leq r_2 \leq k$ e $r_1 + r_2 = k + 1$. Pela hipótese de indução, as duas partes têm, respectivamente, $r_1 - 1$ e $r_2 - 1$ seções, logo a cerca original tem:

$(r_1 - 1) + (r_2 - 1) + 1$ seções

(O extra 1 corresponde à seção que foi anteriormente removida.) Através de aritmética simples, conclui-se que

$r_1 + r_2 - 1 = (k + 1) - 1 = k$ seções.

Este resultado prova que uma cerca com $k+1$ estacas tem k seções, o que verifica $P(k+1)$ e completa a demonstração por indução forte. •



Cerca com 4 estacas e 3 seções

(a)



Cerca com 1 estaca e 0 seções

(b)



Cerca com a última estaca e a última seção removidas

(c)



Cerca com uma seção removida

(d)

Figura 2.2

O exemplo anterior permitiu o uso das duas formas de demonstração por indução porque é possível remover a última parte da cerca ou ainda separá-la em duas partes a partir de um ponto arbitrário. Para alguns problemas a indução forte é a técnica de demonstração mais indicada, sendo que, para alguns dos quais, o seu uso é absolutamente necessário.

EXEMPLO 17 Prove que para todo $n \geq 2$, n é um número primo ou é um produto de números primos.

Usaremos a indução forte; da mesma forma que na indução fraca, a base da indução não precisa começar com o valor 1, e aqui, por motivos óbvios, começaremos pelo valor 2. $P(2)$ é a sentença de que 2 é um número primo ou o produto de dois primos. Como 2 é primo, $P(2)$ é verdadeira. Suponhamos que para todo r , $2 \leq r \leq k$, $P(r)$ é verdadeira — r é primo ou é o produto de números primos. Tomemos agora o número $k+1$. Se $k+1$ é primo, o resultado se verifica. Se $k+1$ não é primo, então ele é um número composto e pode ser escrito como $k+1 = ab$, onde $1 < a < k+1$ e $1 < b < k+1$, ou $2 \leq a \leq k$ e $2 \leq b \leq k$. A hipótese de indução se aplica a a e a b e, portanto, ou a e b são primos ou são produtos de primos. Logo $k+1$ é o produto de números primos. Isto verifica $P(k+1)$ e completa a prova por indução forte. •

EXEMPLO 18 Prove que qualquer valor postal maior ou igual a oito unidades monetárias pode ser obtido usando-se apenas selos com valores de 3 e 5.

Aqui faremos $P(n)$ denotar que apenas selos nos valores de 3 e 5 são necessários para se obter um valor postal n , e provaremos que $P(n)$ é verdadeira para $n \geq 8$. A base da indução consiste em estabelecer $P(8)$, que resulta da equação

$$8 = 3 + 5$$

Por motivos que oportunamente ficarão mais claros, vamos estabelecer dois resultados adicionais, $P(9)$ e $P(10)$ obtidos das equações

$$9 = 3 + 3 + 3$$

$$10 = 5 + 5$$

Vamos assumir então que $P(r)$ é verdadeira para qualquer r , $8 \leq r \leq k$, e examinar o que ocorre com $P(k+1)$. Podemos assumir que $k+1$ é no mínimo 11, posto que já foi mostrado que $P(r)$ é verdadeira para $r = 8, 9$ e 10 . Se $k+1 \geq 11$, vem $(k+1) - 3 = k - 2 \geq 8$, e pela hipótese de indução, $P(k-2)$ é verdadeira ($P(r)$ é verdadeira para $8 \leq r \leq k$). Portanto $k-2$ pode ser escrita como a soma de 3s e 5s, e adicionando-se o valor 3 obtém-se $k-2+3 = k+1$, o que implica que $k+1$ pode ser escrito como a soma de 3s e 5s. Este resultado comprova que $P(k+1)$ é verdadeira e completa a demonstração.

PRÁTICA 8

- Por que os casos $P(9)$ e $P(10)$ são provados separadamente no Exemplo 18?
- Por que não podemos usar indução fraca na demonstração do Exemplo 18?

•

Revisão da Seção 2.2

Técnicas

- Uso da indução fraca em demonstrações
- Uso da indução forte em demonstrações

Idéias Principais

A indução matemática é uma técnica usada para demonstrar propriedades de números inteiros positivos.

Uma demonstração por indução não precisa começar no valor 1.

A indução pode ser usada para demonstrar resultados sobre quantidades, cujos valores são inteiros não-negativos arbitrários.

A indução fraca e a indução forte podem ser usadas para demonstrarem o mesmo resultado; no entanto, dependendo da situação, uma ou outra abordagem pode ser mais fácil de ser utilizada.

hipótese de indução, todos têm o mesmo fabricante. Então o fabricante de HAL é o mesmo dos outros computadores, o que prova que os $k+1$ computadores têm o mesmo fabricante.

48. Uma pitoresca tribo nativa tem apenas três palavras na sua língua, *cuco*, *cuca* e *caco*. Novas palavras são compostas pela concatenação destas palavras em qualquer ordem, por exemplo *cucacucocacocuca*. Use a indução completa (para o número de subpalavras na palavra) para provar que qualquer palavra nesta língua tem um número par de c 's.
- ★49. Demonstre que qualquer valor postal maior ou igual a duas unidades monetárias pode ser obtido usando-se somente selos com valor de 2 e 3.
50. Prove que qualquer valor postal maior ou igual a 12 pode ser obtido usando-se somente selos com valores de 4 e 5.
51. Prove que qualquer valor postal maior ou igual a 14 unidades monetárias pode ser obtido usando-se apenas selos de 3 e 8.
- ★52. Prove que qualquer valor postal maior ou igual a 64 unidades monetárias pode ser obtido usando-se somente selos de 5 e 17.
53. Em qualquer grupo de k pessoas, $k \geq 1$, cada um deve apertar a mão de todas as outras pessoas. Encontre uma fórmula que forneça o número de apertos de mão, e demonstre-a como verdadeira, usando indução.

Seção 2.3 Recursão e Relação de Recorrência

Definições Recursivas

Uma definição na qual o item que está sendo definido aparece como parte da definição é chamada **definição indutiva** ou **definição recursiva**. À primeira vista isto pode parecer sem sentido — como algo pode ser definido em termos dele próprio? Este procedimento funciona porque as definições recursivas são compostas de duas partes:

1. uma base, onde alguns casos simples do item que está sendo definido são dados explicitamente, e
2. um passo indutivo ou recursivo, onde outros casos do item que está sendo definido são dados em termos dos casos anteriores.

A parte 1 nos fornece um ponto de partida na medida em que trata alguns casos simples; enquanto a parte 2 nos permite construir novos casos a partir desses casos simples, para então construir outros casos a partir desses novos, e assim por diante. (A analogia com demonstrações por indução matemática justifica o nome "definição indutiva". Em uma demonstração por indução, existe a base da indução, isto é, a demonstração de $P(1)$ — ou a demonstração de P para algum outro valor inicial — e existe a hipótese indutiva, onde a validade de $P(k+1)$ é deduzida da validade de P para valores menores.)

A recursão é uma idéia importante que pode ser utilizada para definir seqüências de objetos, coleções mais gerais de objetos e operações sobre objetos. (O Predicado Prolog *na-cadeia-alimentar* da Seção 1.5 foi definido recursivamente.) Até mesmo algoritmos podem ser definidos recursivamente.

Seqüências Recursivas

Uma **seqüência** S é uma lista de objetos que são enumerados segundo alguma ordem; existe um primeiro objeto, um segundo, e assim por diante. $S(k)$ denota o k -ésimo objeto da seqüência. Uma seqüência é definida recursivamente, explicitando-se seu primeiro valor (ou seus primeiros valores) e, a partir daí, definindo-se outros valores na seqüência em termos dos valores iniciais.

EXEMPLO 19 A seqüência S é definida recursivamente por:

1. $S(1) = 2$
2. $S(2) = 2S(n-1)$ para $n \geq 2$

Pela afirmação 1, $5(1)$, o primeiro objeto em S , é 2. Então pela sentença 2, o segundo objeto em S é $5(2) = 25(1) = 2(2) = 4$. Novamente por 2, $S(3) = 2S(2) = 2(4) = 8$. Dessa forma podemos deduzir que S é a sequência

$$2, 4, 8, 16, 32, \dots$$

Uma regra semelhante à da sentença 2 no Exemplo 19, na qual se define um valor da sequência a partir de um ou mais valores anteriores, é chamada **relação de recorrência**.

PRÁTICA 9 A sequência T é definida recursivamente como se segue:

1. $T(1) = 1$
2. $T(n) = T(n-1) + 3$ para $n \geq 2$

Escreva os primeiros cinco valores da sequência T .

EXEMPLO 20 A famosa **seqüência de Fibonacci** é definida recursivamente por:

$$\begin{aligned} F(1) &= 1 \\ F(2) &= 2 \\ F(n) &= F(n-2) + F(n-1) \text{ para } n > 2 \end{aligned}$$

Aqui os primeiros dois valores na sequência são dados, e a relação de recursão define o n -ésimo termo a partir dos dois anteriores.

PRÁTICA 10 Escreva os oito primeiros valores da sequência de Fibonacci.

A sequência de Fibonacci tem sido estudada exaustivamente. Por ter sido definida de forma recursiva, podemos usar a indução para provar muitas de suas propriedades.

EXEMPLO 21 Prove que na sequência de Fibonacci,

$$F(n+4) = 3F(n+2) - F(n) \text{ para todo } n \geq 1.$$

Para a base da indução, devemos mostrar dois casos, $n = 1$ e $n = 2$. Para $n = 1$, temos:

$$F(5) = 3F(3) - F(1)$$

ou (usando valores obtidos na Prática 10)

$$5 = 3(2) - 1$$

que é verdadeiro. Para $n = 2$,

$$F(6) = 3F(4) - F(2)$$

ou

$$8 = 3(3) - 1$$

que também é verdadeiro. Suponhamos agora que para todo r , $1 \leq r \leq k$,

$$F(r+4) = 3F(r+2) - F(r).$$

Agora, mostremos o caso $k+1$, onde $k+1 \geq 3$. Então, desejamos mostrar que:

$$F(k+1+4) \stackrel{?}{=} 3F(k+1+2) - F(k+1)$$

ou

$$F(k+5) \stackrel{?}{=} 3F(k+3) - F(k+1)$$

Da relação de recorrência na sequência de Fibonacci temos:

$$F(k+5) = F(k+3) + F(k+4)$$

e, pela hipótese de indução, para $r = k-1$ e $r = k$, respectivamente, vem:

$$F(k+3) = 3F(k+1) - F(k-1)$$

e

$$F(k+4) = 3F(k+2) - F(k)$$

Logo

$$\begin{aligned} F(k+5) &= 3F(k+1) - F(k-1) + 3F(k+2) - F(k) \\ &= 3[F(k+1) - F(k+2)] - [F(k-1) - F(k)] \\ &= 3F(k+3) - F(k+1) \text{ (usando novamente a relação de recorrência)} \end{aligned}$$

A indução completa foi necessária aqui, porque a relação de recorrência para a sequência de Fibonacci usa mais termos do que o termo imediatamente anterior. •

PRÁTICA 11 Na demonstração do Exemplo 21, por que é necessário provar $n = 2$ como um caso especial? •

Conjuntos Recursivos

Os objetos de uma sequência são ordenados — existe um primeiro objeto, um segundo, e assim por diante. Um *conjunto* de objetos é uma coleção de objetos na qual nenhuma regra de ordenação é imposta. Alguns conjuntos podem ser definidos recursivamente.

EXEMPLO 22 Na Seção 1.1 notamos que certas cadeias de proposições, conectivos lógicos e parênteses tais como $(A \wedge B)' \vee C$, são considerados válidos, enquanto outras cadeias como $\wedge \wedge A''B$ não são válidas. A sintaxe para ordenar estes símbolos constitui a definição do conjunto de fórmulas proposicionais bem-formuladas, que é uma definição recursiva.

1. Qualquer proposição é uma wff.
2. Se P e Q são wffs, então também o serão $(P \wedge Q)$, $(P \vee Q)$, $(P \rightarrow Q)$, (P') e $(P \leftrightarrow Q)$.

Nós normalmente omitimos parênteses quando isso não nos causa confusão; então escrevemos $(P \vee Q)$ como $P \vee Q$, ou (P') como P' . Iniciando com uma proposição e usando a regra 2 repetidamente, podemos construir qualquer wff proposicional. Por exemplo, A, B , e C são wffs pela regra 1. Pela regra 2,

$$(A \wedge B) \text{ e } (C')$$

são também wffs. Novamente pela regra 2,

$$[(A \wedge B) \rightarrow (C')]$$

é uma wff. Aplicando a regra 2 mais uma vez, obtém-se a wff

$$([(A \wedge B) \rightarrow (C')])'$$

Eliminando alguns parênteses, podemos escrever a wff como

$$[(A \wedge B) \rightarrow C']'$$

PRÁTICA 12 Mostre como construir a wff $[(A \vee (B')) \rightarrow C]$ a partir da definição do Exemplo 22. •

PRÁTICA 13 Uma definição recursiva para o conjunto de pessoas que são ancestrais de João pode ser construída a partir da seguinte premissa:

Os pais de João são ancestrais de João

Forneça a base da indução. •

Cadeias de símbolos obtidas a partir de um "alfabeto" finito são objetos normalmente encontrados na ciência da computação. Os computadores armazenam dados em **cadeias binárias**, ou seja, cadeias do alfabeto consistindo em 0s e 1s; os compiladores entendem as instruções de um programa como cadeias de *tokens*, tais como palavras-chave e identificadores. A coleção de todas as cadeias de tamanho finito de um alfabeto, normalmente chamadas de cadeias *sobre* um alfabeto, podem ser definidas recursivamente (veja o próximo exemplo). Muitos conjuntos de cadeias de caracteres com propriedades especiais têm também definições recursivas.

EXEMPLO 23 O conjunto de todas as cadeias de símbolos (de tamanho finito) sobre um alfabeto A é denotado por A^* . A definição recursiva de A^* é:

1. A **cadeia** vazia Λ (a cadeia sem símbolos) pertence a A^* .
2. Qualquer elemento de A pertence a A^* .
3. Se x e y são cadeias em A^* , então a **concatenação** xy também pertence a A^* .

As partes 1 e 2 constituem a base da recursão, e a parte 3 é o passo recursivo desta definição. Perceba que para qualquer cadeia x , $x\Lambda = \Lambda x = x$. •

PRÁTICA 14 Se $x = 1011$ e $y = 001$, escreva as cadeias xy , yx , e $yx\Lambda x$. •

PRÁTICA 15 Forneça uma definição recursiva para o conjunto de todas as cadeias binárias que são **palíndromos**, ou seja, cadeias iguais, se lidas do início para o fim ou ao contrário. •

EXEMPLO 24 Suponha que em uma certa linguagem de programação, os identificadores podem ser cadeias alfanuméricas de tamanho arbitrário, mas que devem começar com uma letra. Uma definição recursiva para um conjunto de cadeias desta natureza é:

1. Uma letra é um identificador.
2. Se A é um identificador, então a concatenação de A com qualquer letra ou dígito também é um identificador.

Uma notação mais simbólica para descrever conjuntos de cadeias que são definidas recursivamente é chamada **forma de Backus Naur**, ou **BNF**, originalmente desenvolvida para definir a linguagem de programação ALGOL. Na notação BNF, itens que são definidos em termos de outros itens são envolvidos por menor e maior ($< >$), enquanto itens específicos que não são definidos mais adiante não aparecem entre menor e maior. A linha vertical $|$ indica uma escolha com o mesmo significado da palavra *ou*. A definição BNF de um identificador é:

$$\begin{aligned}\langle \text{identificador} \rangle &::= \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{letra} \rangle \mid \langle \text{identificador} \rangle \langle \text{dígito} \rangle \\ \langle \text{letra} \rangle &::= a \mid b \mid c \mid \dots \mid z \\ \langle \text{dígito} \rangle &::= 0 \mid 1 \mid \dots \mid 9\end{aligned}$$

Então o identificador *mel* é obtido da definição por uma sequência de escolhas tais como:

$$\begin{aligned}\langle \text{identificador} \rangle &\text{ pode ser } \langle \text{identificador} \rangle \langle \text{dígito} \rangle \\ &\text{ que pode ser } \langle \text{identificador} \rangle^2 \\ &\text{ que pode ser } \langle \text{identificador} \rangle \langle \text{letra} \rangle^2 \\ &\text{ que pode ser } \langle \text{identificador} \rangle e^2 \\ &\text{ que pode ser } \langle \text{letra} \rangle e^2 \\ &\text{ que pode ser } me^2\end{aligned}$$

Operações Recursivas

Podemos definir recursivamente certas operações sobre objetos, como nos exemplos abaixo.

EXEMPLO 25 Uma definição recursiva para a operação de exponenciação a^n , definida para um número real a diferente de zero e n inteiro não-negativo é:

1. $a^0 = 1$
2. $a^n = (a^{n-1})a$ para $n \geq 1$

EXEMPLO 26 Uma definição recursiva para a multiplicação de dois inteiros positivos m e n é:

1. $m(1) = m$
2. $m(n) = m(n-1) + m$ para $n \geq 2$

PRÁTICA 16 Seja x uma cadeia sobre algum alfabeto. Forneça uma definição recursiva para a operação x^n (concatenação de x com ele próprio n vezes) para $n \geq 1$.

Na Seção 1.1, definimos a operação de disjunção lógica de duas proposições. Isso pode servir como a base da recursão para a definição recursiva da disjunção de n proposições, para $n \geq 2$:

1. $A_1 \vee A_2$ definida como na Seção 1.1
2. $A_1 \vee \cdots \vee A_n = (A_1 \vee \cdots \vee A_{n-1}) \vee A_n$ para $n > 2$

(D)

Usando esta definição, podemos generalizar a propriedade associativa da disjunção (equivalência tautológica 2a) para afirmar que em uma disjunção de n proposições, o agrupamento por parênteses é desnecessário, porque todos esses agrupamentos são equivalentes à expressão geral da disjunção de n proposições. Simbolicamente, para qualquer n , com $n \geq 3$ e qualquer p , com $1 \leq p \leq n-1$,

$$(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_n) \Leftrightarrow A_1 \vee \cdots \vee A_n$$

Esta equivalência pode ser provada por indução em n . Para $n = 3$,

$$\begin{aligned} A_1 \vee (A_2 \vee A_3) &\Leftrightarrow (A_1 \vee A_2) \vee A_3 && \text{(pela equivalência 2a)} \\ &= A_1 \vee A_2 \vee A_3 && \text{(pela equação (1))} \end{aligned}$$

Suponha que para $n = k$ e $1 \leq p \leq k-1$

$$(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_k) \Leftrightarrow A_1 \vee \cdots \vee A_k$$

Então para $1 \leq p \leq k$

$$\begin{aligned} &(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_{k+1}) \\ &= (A_1 \vee \cdots \vee A_p) \vee [(A_{p+1} \vee \cdots \vee A_k) \vee A_{k+1}] && \text{(pela equação (1))} \\ &\Leftrightarrow [(A_1 \vee \cdots \vee A_p) \vee (A_{p+1} \vee \cdots \vee A_k)] \vee A_{k+1} && \text{(pela equivalência 2a)} \\ &\Leftrightarrow (A_1 \vee \cdots \vee A_k) \vee A_{k+1} && \text{(pela hipótese de indução)} \\ &= A_1 \vee \cdots \vee A_{k+1} && \text{(pela equação (1))} \end{aligned}$$

Algoritmos Recursivos

O Exemplo 19 fornece a definição recursiva de uma sequência S . Suponha que desejamos escrever um programa de computador para obter $S(n)$ para algum inteiro positivo n . Podemos utilizar duas abordagens diferentes. Se desejamos encontrar $S(12)$, por exemplo, podemos começar com $S(1) = 2$ e então computar $S(2)$, $S(3)$, e assim por diante, como fizemos no Exemplo 19, até que finalmente obtemos $S(12)$. Esta abordagem, sem dúvida, envolve iterações através de uma espécie de laço. Uma versão desse *algoritmo iterativo* é mostrada abaixo, implementada como uma função em Pascal.

```

function  $S(n$ : integer): integer;
  {cálculo iterativo do valor  $S(n)$  para a seqüência de Exemplo 19}

  var
    i: integer;           {índice do loop}
    ValorCorrente: integer; {valor corrente da função}

  begin
    if  $n = 1$  then
       $S := 2$ 
    else
      begin
         $i := 2$ ;
         $ValorCorrente := 2$ ;
        while  $i \leq n$  do
          begin
             $ValorCorrente := 2 * ValorCorrente$ ;
             $i := i + 1$ ;
          end;

          {o ValorCorrente tem agora o valor  $S(n)$ }
           $S := ValorCorrente$ ;
        end;
      end;
    end;
  end;

```

A base, quando $n = 1$, é obtida na cláusula **then** da instrução **if**. A cláusula **else**, para $n > 1$, realiza uma inicialização e, a seguir, entra no laço **while**, que calcula os demais valores da seqüência, até que o limite superior da mesma é alcançado. Você pode acompanhar a execução passo a passo deste algoritmo para valores pequenos de n , de forma a se convencer de que ele funciona.

A segunda abordagem utilizada para calcular $S(n)$ usa a definição de recursão de 5 diretamente. Uma versão do *algoritmo recursivo*, implementado novamente em Pascal, é apresentada a seguir.

```

function  $S(n$  : integer):integer;
  {Calcula recursivamente os valores de  $S(n)$  para a seqüência do Exemplo 19}

  begin
    if  $n = 1$  then
       $S := 2$ 
    else
       $S := 2 * S(n-1)$ ;
    end;
  end;

```

O corpo desta função consiste em uma única estrutura **if-then-else**. Para entendermos o seu funcionamento, vamos acompanhar a execução da mesma para calcular $S(3)$. A função é inicialmente chamada para o valor de entrada $n = 3$. Como n é diferente de 1, a execução é direcionada para a cláusula **else**. Neste ponto o procedimento para calcular $S(3)$ é momentaneamente suspenso, até que o valor de $S(2)$ seja conhecido. Qualquer informação relevante para a obtenção de $S(3)$ é armazenada na memória do computador numa *pilha*, para ser posteriormente carregada quando o cálculo puder ser completado. (Uma pilha é uma coleção de dados onde qualquer novo item ao entrar é armazenado no topo, e somente o item do topo da pilha pode ser removido ou acessado. Uma pilha é então uma estrutura do tipo *LIFO* - last in, first out.) A função é novamente chamada para um valor de entrada $n = 2$. Novamente a cláusula **else** é executada e o cálculo de $S(2)$ é momentaneamente suspenso com as informações relevantes armazenadas na pilha, enquanto a função é chamada novamente, agora com $n = 1$ como entrada.

Desta vez o valor de retorno, 2, pode ser calculado diretamente pela cláusula **then** da função. Esta última chamada à função retorna este valor à penúltima chamada, que pode recuperar qualquer informação relevante para o caso $n = 2$ da pilha, calcular $S(2)$ e disponibilizar o resultado à chamada anterior (a primeira chamada). Finalmente, esta chamada inicial de S é capaz de esvaziar a pilha e concluir seus cálculos, retornando o valor de $S(3)$.

Quais são as vantagens e desvantagens de algoritmos iterativo e recursivo para executar uma determinada tarefa? Neste exemplo, a versão recursiva é menor, pois dispensa o gerenciamento do laço. Descrever a

execução de uma versão recursiva é mais complexo do que descrever a versão iterativa, porém todos os passos são realizados automaticamente. Não precisamos saber o que está acontecendo internamente, é necessário apenas termos conhecimento de que uma quantidade grande de chamadas pode demandar muita memória para o armazenamento na pilha das informações necessárias às chamadas anteriores. Se for necessária mais memória do que o que se tem disponível, ocorre um "estouro de pilha". Além de usar muita memória, algoritmos recursivos podem demandar muitos outros procedimentos computacionais e, por isso, podem ser mais lentos do que os não-recursivos (ver Exercício 7, Seção No Computador, no fim deste capítulo).

De alguma forma, a recursão fornece uma maneira natural de se abordar uma grande variedade de problemas, alguns dos quais poderiam ter soluções não-recursivas extremamente complexas. Problemas onde se desejam calcular valores para uma seqüência definida recursivamente são naturalmente escritos em algoritmos recursivos. Muitas linguagens de programação permitem o uso de recursão (infelizmente, nem todas).

PRÁTICA 17 Escreva o corpo da função recursiva para computar $T(n)$ para a seqüência T definida na Prática 9. •

EXEMPLO 27 No Exemplo 26, uma definição recursiva foi dada para multiplicar dois inteiros positivos m e n . Uma versão em Pascal de um algoritmo recursivo para multiplicação baseada nesta definição é dada abaixo:

```
function Produto( $m,n$ :integer):integer;
{calcula recursivamente o produto de  $m$  por  $n$ }

begin
    if  $n = 1$  then
        Produto :=  $m$ 
    else
        Produto := Produto( $m, n - 1$ ) +  $m$ ;
end;
```

Um algoritmo recursivo chama ele próprio para valores "menores" que o valor de entrada. Suponha um problema que pode ser resolvido pela solução de versões menores do mesmo problema, e que essas versões tornam-se, em algum momento, casos triviais que podem ser manipulados facilmente. Então, um algoritmo recursivo pode ser útil, mesmo que o problema original não seja colocado de forma recursiva.

Para nos convenceremos de que um algoritmo recursivo funciona não precisamos começar com um particular valor de entrada, e a partir daí trabalhar com os valores anteriores até alcançarmos o caso trivial e então voltar para calcular o valor inicial. Acompanhamos este procedimento para computar o valor de $S(3)$ apenas para ilustrar o mecanismo computacional de recursividade. Em vez disso, nós poderíamos verificar o caso trivial (da mesma forma que verificamos o passo básico em uma prova por indução) e verificar que *se* o algoritmo funciona corretamente quando é chamado para valores de entrada menores, então ele resolve o problema para o valor original de entrada (este fato é semelhante a provar $P(k+1)$ a partir da hipótese $P(k)$ em uma prova por indução).

EXEMPLO 28 Uma das tarefas mais comuns em processamento de dados é ordenar uma lista L de n termos em ordem crescente ou decrescente numérica ou alfabeticamente. (A lista pode consistir, por exemplo, em nomes de clientes, e na ordenação "Zé das Couves" deve vir depois de "João da Silva".)

```
ALGORITMO SelectionSort

procedure SelectionSort( $L$ : lista;  $j$ : integer);
{ordenação recursiva crescente dos itens de 1 até  $j$  da lista  $L$ }

begin
    if  $j = 1$  then
        ordenação está completa, imprima a lista ordenada
    else
        begin
            encontre o índice  $i$  do maior elemento de  $L$  entre 1 e  $j$ ;
            troque  $L(i)$  com  $L(j)$ ;
            SelectionSort( $L, j - 1$ );
        end;
    end;
```

O algoritmo de **selectionsort** — um algoritmo simples, mas pouco eficiente — é descrito em pseudocódigo no quadro acima. Ele ordena os primeiros j itens de uma lista L em ordem crescente; quando o procedimento é inicialmente chamado j tem o valor n (então a primeira chamada, em última análise, ordena a lista inteira). A parte recursiva do algoritmo reside na cláusula **else**; o algoritmo examina a seção da lista que está sendo considerada e determina i de tal forma que $L(i)$ seja o maior valor. O algoritmo troca então $L(i)$ e $L(j)$, depois disso o máximo valor ocorre na posição j , ou seja, na última posição da seção considerada da lista. $L(j)$ agora está correto e não deve mais ser alterado, este processo é repetido para a lista $L(i)$ a $L(j-1)$. Se esta parte da lista for ordenada corretamente, então a lista inteira também estará corretamente ordenada. Tão logo j assumo o valor 1 (a cláusula **then**) a parte da lista submetida à ordenação consistirá em uma única entrada, a qual certamente estará no lugar correto. Neste ponto a lista inteira estará ordenada. •

EXEMPLO 29

Agora que ordenamos nossa lista, outra tarefa comum é procurar se um particular item ocorre na lista. (Será que João da Silva é um cliente?) Uma técnica eficiente de busca para uma lista ordenada é o **algoritmo de busca binária** recursivo que é descrito em pseudocódigo no quadro a seguir. O algoritmo procura o valor x na seção da lista L entre $L(i)$ e $L(j)$; inicialmente i e j têm os valores 1 e n , respectivamente. A cláusula **then** do primeiro **if** é a base da recursão que determina que x não pode ser encontrado em uma lista vazia, ou seja, uma lista onde o primeiro índice excede o último. Na cláusula **else**, devemos obter o item mediano da seção considerada da lista. (Se a seção contém um número ímpar de itens, existe realmente um item mediano; se a seção contém um número par de itens, é suficiente tomar como mediano o último item da primeira metade da seção em questão.) Comparando x com o valor mediano, ou localizamos x ou determinamos em qual metade da lista devemos continuar a busca. •

ALGORITMO BuscaBinária

```

procedure BuscaBinária( $L$ : lista;  $i, j$ : integer;  $x$ : tipoitem);
{procura o item  $x$  em uma lista  $L$  entre  $L(i)$  e  $L(j)$ }

begin
  if  $i > j$  then
    write('não encontrado')
  else
    begin
      encontre o índice  $k$  do item mediano na lista  $L(i) \text{ — } L(j)$ ;
      if  $x = \text{item mediano}$  then
        write('encontrado')
      else
        if  $x < \text{item mediano}$  then
          BuscaBinária ( $L, i, k-1, x$ )
        else
          BuscaBinária ( $L, k+1, j, x$ );
        end;
    end;
end;

```

EXEMPLO 30

Suponha uma lista com os seguintes números:

3, 7, 8, 10, 14, 18, 22, 34

e o valor de busca x igual a 25. A lista inicial é não-vazia, logo podemos localizar e determinar o item mediano que, no caso, é 10. Comparamos então x com o item mediano. Como $x > 10$, a busca é realizada na segunda metade da lista, ou seja

14, 18, 22, 34

Aqui novamente a lista é não-vazia, e o valor mediano é 18. Como $x > 18$ a busca é realizada na segunda metade da lista, ou seja,

22, 34

Para esta lista não-vazia, o valor mediano é 22. Como $x > 22$, a busca continua na segunda metade da lista, ou seja

34

Esta é uma lista com um único elemento, com o valor mediano igual a este elemento. Como $x < 34$ a busca deve começar na "primeira metade" da lista que, no caso, está vazia. Neste ponto o algoritmo é encerrado informando que o valor procurado x não está na lista.

Este procedimento exigiu ao todo quatro comparações, x foi comparado, a cada momento, a 10, 18, 22 e 34. •

PRÁTICA 18

Na busca binária da lista no Exemplo 30, obtenha os valores com os quais x é comparado, caso x assumo o valor 8. •

Resolvendo Relações de Recorrência

Para obtermos o valor $S(n)$ da sequência S do Exemplo 19, desenvolvemos dois algoritmos, um iterativo e outro recursivo. No entanto, existe ainda uma forma mais fácil para computar $S(n)$. Lembremos que

$$S(1) = 2 \quad (1)$$

$$S(n) = 2S(n-1) \quad \text{para } n \geq 2 \quad (2)$$

Como

$$S(1) = 2 = 2^1$$

$$S(2) = 4 = 2^2$$

$$S(3) = 8 = 2^3$$

$$S(4) = 16 = 2^4$$

e assim por diante, podemos concluir que:

$$S(n) = 2^n \quad (3)$$

Usando a equação (3), podemos escolher um valor para n e computar $S(n)$ sem a necessidade de computarmos inicialmente S para valores menores que n . Uma equação como a (3), onde podemos substituir um valor e obter diretamente o seu resultado, é chamada **solução de forma fechada** para a relação de recorrência (2) sujeita à hipótese básica (base da recorrência) estabelecida em (1). Encontrar uma solução de forma fechada chama-se **resolver** a relação de recorrência. Obviamente, é interessante encontrar uma solução de forma fechada sempre que for possível.

Uma técnica utilizada para resolver relações de recorrência é uma abordagem do tipo "expanda, suponha e verifique", que usa repetidamente a relação de recorrência para expandir a expressão para o n -ésimo termo, até que um padrão geral de comportamento da expressão possa ser deduzido. Finalmente, a suposição é verificada (demonstrada) por indução matemática.

EXEMPLO 31

Consideremos novamente a hipótese básica e a relação de recorrência para a sequência S do Exemplo 19:

$$S(1) = 2 \quad (4)$$

$$S(n) = 2S(n-1) \quad \text{para } n \geq 2 \quad (5)$$

Vamos supor que ainda não conhecemos a solução de forma fechada, e usar a abordagem de expandir, supor e verificar para encontrá-la. Começando com $S(n)$, expandiremos pelo uso repetido da relação de recorrência. Lembrando que a relação de recorrência é a fórmula que diz que qualquer valor de S pode ser substituído por duas vezes o valor anterior de S . Aplicaremos esta fórmula a S para os valores $n-1$, $n-2$, e assim por diante:

$$\begin{aligned} S(n) &= 2S(n-1) \\ &= 2[2S(n-2)] = 2^2S(n-2) \\ &= 2^2[2S(n-3)] = 2^3S(n-3) \\ &\vdots \\ &\vdots \\ &\vdots \end{aligned}$$

Observando a evolução do padrão, podemos supor que após k expansões sucessivas, a equação terá a forma:

$$S(n) = 2^k S(n-k)$$

Esta expansão dos valores de S em termos dos valores inferiores de S deve parar quando $n - k = 1$, ou seja, quando $k = n - 1$. Neste ponto, teremos

$$\begin{aligned} S(n) &= 2^{n-1} S[n - (n - 1)] \\ &= 2^{n-1} S(1) = 2^{n-1}(2) = 2^n \end{aligned}$$

que é a expressão da solução de forma fechada.

Ainda não terminamos, pois o que fizemos até agora foi supor encontrar um padrão geral. Devemos confirmar agora nossa solução de forma fechada utilizando indução em n . A afirmação que desejamos demonstrar é, portanto, $S(n) = 2^n$ para $n \geq 1$.

Para a hipótese básica, $S(1) = 2^1$. Este resultado é verdadeiro pela equação (4). Suponhamos então que $S(k) = 2^k$. Temos então:

$$\begin{aligned} S(k+1) &= 2 S(k) && \text{(pela equação (5))} \\ &= 2 (2^k) && \text{(pela hipótese indutiva)} \\ &= 2^{k+1} \end{aligned}$$

Este resultado prova que a nossa solução de forma fechada está correta. •

PRÁTICA 19

Encontre uma solução de forma fechada para a relação de recorrência, sujeita à hipótese básica para a sequência T :

1. $T(1) = 1$
2. $T(n) = T(n-1) + 3$ para $n \geq 2$

(Dica: Expanda, suponha e verifique.) •

Os métodos usados na solução de relação de recorrência são semelhantes àqueles usados para resolver equações diferenciais. Em particular, relações de recorrência e equações diferenciais são classificadas em vários tipos, muitos dos quais têm fórmulas de solução que são conhecidas.

Uma relação de recorrência para a sequência $S(n)$ é **linear** se valores anteriores de S que aparecem na definição ocorrem apenas à primeira potência. A relação de recorrência linear mais geral tem a forma:

$$S(n) = f_1(n)S(n-1) + f_2(n)S(n-2) + \dots + f_k(n)S(n-k) + g(n)$$

onde os f_i 's e g podem ser expressões envolvendo n . A relação de recorrência tem **coeficientes constantes** se os f_i 's são todos constantes. Ela é de **primeira ordem** se o n -ésimo termo depende apenas dos termos de ordem $n-1$. Uma relação de recorrência de primeira ordem com coeficientes constantes tem a forma

$$S(n) = cS(n-1) + g(n) \tag{6}$$

Finalmente, uma relação de recorrência é **homogênea** se $g(n) = 0$ para todo n .

Encontraremos a solução de forma fechada da equação (6), a relação de recorrência linear de primeira ordem com coeficientes constantes, sujeita à hipótese básica (base da recorrência) $S(1)$, que é conhecida. Usaremos a abordagem de expandir, supor e verificar. O procedimento a ser implementado aqui é uma generalização do que foi feito no Exemplo 31. Aplicando-se repetidamente a equação (6) e simplificando, obtém-se:

$$\begin{aligned} S(n) &= cS(n-1) + g(n) \\ &= c[cS(n-2) + g(n-1)] + g(n) \\ &= c^2S(n-2) + cg(n-1) + g(n) \\ &= c^2[cS(n-3) + g(n-2)] + cg(n-1) + g(n) \\ &= c^3S(n-3) + c^2g(n-2) + cg(n-1) + g(n) \\ &\vdots \\ &\vdots \end{aligned}$$

Após k expansões sucessivas, a fórmula geral parece ser:

$$S(n) = c^k S(n-k) + c^{k-1} g(n-(k-1)) + \dots + cg(n-1) + g(n).$$

Se a sequência tem o valor base igual a 1, então a expansão termina quando $n - k = 1$ ou $k = n - 1$. Neste ponto teremos:

$$\begin{aligned} S(n) &= c^{n-1}S(1) + c^{n-2}g(2) + \dots + cg(n-1) + g(n) \\ &= c^{n-1}S(1) + c^{n-2}g(2) + \dots + c^1g(n-1) + c^0g(n) \end{aligned} \quad (7)$$

Podemos usar a **notação de somatório** para escrever parte desta expressão de forma mais compacta. A letra grega maiúscula sigma, $\sum$, representa o somatório. A notação $\sum_{i=p}^q (\text{expressão})$ significa que devemos substituir na expressão sucessivos valores de i , o **índice do somatório**, do limite inferior p até o limite superior q , e então somar os resultados obtidos. (Veja Apêndice A mais adiante, para discussões sobre a notação do somatório.)

Então, por exemplo,

$$\sum_{i=1}^7 a(i) = a(1) + a(2) + \dots + a(7)$$

Na notação de somatório a equação (7) será:

$$S(n) = c^{n-1} S(1) + \sum_{i=2}^n c^{n-i} g(i)$$

Podemos usar a indução de forma mais explícita do que a utilizada no Exemplo 31 para verificar que esta fórmula funciona (veja Exercício 67). Entretanto, a solução de forma fechada para a relação de recorrência (6) é

$$S(n) = c^{n-1} S(1) + \sum_{i=2}^n c^{n-i} g(i) \quad (8)$$

EXEMPLO 32 A sequência $S(n)$ do Exemplo 31,

$$\begin{aligned} S(1) &= 2 \\ S(n) &= 2 S(n-1) \quad \text{para } n \geq 2 \end{aligned}$$

é uma relação de recorrência linear, homogênea, de primeira ordem com coeficientes constantes. Em outras palavras, ela verifica a equação (6) com $c = 2$ e $g(n) = 0$. Da fórmula (8), a solução de forma fechada é

$$S(n) = 2^{n-1}(2) + \sum_{i=2}^n 0 = 2^n$$

o que confirma o resultado previamente obtido no Exemplo 31. •

PRÁTICA 20 Refaça a Prática 19, usando a equação (8). •

Revisão da Seção 2.3

Técnicas

- Geração de valores em uma sequência definida recursivamente.
- Uso da indução para demonstrar propriedades de uma sequência definida recursivamente.
- Reconhecimento de objetos de uma coleção definida recursivamente.
- Como fornecer definições recursivas para determinados conjuntos de objetos.
- Como fornecer definições recursivas para certas operações sobre objetos.
- Como escrever algoritmos recursivos para gerar sequências definidas recursivamente.
- Resolução de relações de recorrência lineares de primeira ordem com coeficientes constantes, usando uma fórmula de solução.
- Resolução de relações de recorrência através da técnica de expansão, suposição e verificação.

Idéias Principais

Podemos usar definições recursivas para seqüências de objetos, conjuntos de objetos e operações sobre objetos, desde que informações básicas sejam conhecidas e novas informações dependam de informações já conhecidas.

Os algoritmos recursivos fornecem uma forma natural de resolver certos problemas, através da utilização da mesma tarefa para solucionar uma versão reduzida do problema.

Certas relações de recorrência têm soluções de forma fechada.

Exercícios 2.3

Para os Exercícios 1 a 10, escreva os cinco primeiros valores das seqüências dadas.

★1. $S(1) = 10$

$$S(n) = S(n-1) + 10 \quad \text{para } n \geq 2$$

2. $A(1) = 2$

$$A(n) = \frac{1}{A(n-1)} \quad \text{para } n \geq 2$$

3. $B(1) = 1$

$$B(n) = B(n-1) + n^2 \quad \text{para } n \geq 2$$

★4. $S(1) = 1$

$$S(n) = S(n-1) + \frac{1}{n} \quad \text{para } n \geq 2$$

5. $T(1) = 1$

$$T(n) = nT(n-1) \quad \text{para } n \geq 2$$

6. $P(1) = 1$

$$P(n) = n^2P(n-1) + (n-1) \quad \text{para } n \geq 2$$

★7. $M(1) = 2$

$$M(2) = 2$$

$$M(n) = 2M(n-1) + M(n-2) \quad \text{para } n > 2$$

8. $D(1) = 3$

$$D(2) = 5$$

$$D(n) = (n-1)D(n-1) + (n-2)D(n-2) \quad \text{para } n \geq 2$$

9. $W(1) = 2$

$$W(2) = 3$$

$$W(n) = W(n-1)W(n-2) \quad \text{para } n > 2$$

10. $T(1) = 1$

$$T(2) = 2$$

$$T(3) = 3$$

$$T(n) = T(n-1) + 2T(n-2) + 3T(n-3) \quad \text{para } n > 3$$

Nos Exercícios 11 e 12, prove as propriedades dadas para os números de Fibonacci diretamente da definição. (Não é necessário usar indução.)

★11. $F(n+1) + F(n-2) = 2F(n) \quad \text{para } n > 2$

12. $[F(n+1)]^2 = [F(n)]^2 + F(n-1)F(n+2) \quad \text{para } n > 1$

Nos Exercícios 13 a 18, demonstre as propriedades dadas para os números de Fibonacci para todo $n \geq 1$.

23. Prove a correção da função iterativa da Seção 2.3 para calcular $S(n) = 2^n$.

Revisão do Capítulo 2

Terminologia

algoritmo de busca binária	índice do somatório	raciocínio indutivo
algoritmo de busca seqüencial	indução completa	recíproca
algoritmo selectionsort	indução forte	regra de inferência de loop
algoritmos dividir para conquistar	indução fraca	relação de recorrência
base da indução	invariante de loop	relação de recorrência com coeficientes constantes
cadeia binária	notação de somatório	relação de recorrência de primeira ordem
cadeia vazia	número racional	relação de recorrência
concatenação	palíndromo	homogênea
contra-exemplo	passo indutivo	relação de recorrência linear
contrapositiva	princípio da boa ordenação	seqüência
correção parcial	princípio da indução	seqüência de Fibonacci
definição indutiva	matemática	solução de forma fechada
definição recursiva	prova direta	solução de uma relação de recorrência
Fórmula de Backus Naur (BNF)	prova por contradição	suposição indutiva
hipótese indutiva	prova por contraposição	
	prova por exaustão	
	raciocínio dedutivo	

Avaliação

Sem consultar o capítulo, responda verdadeiro ou falso para as seguintes questões.

Seção 2.1

1. Um teorema nunca poderá ser demonstrado simplesmente, provando-se que é verdadeiro para um número finito de casos.
2. A prova por contradição de $P \rightarrow Q$ é iniciada pela suposição de P e Q' .
3. No enunciado do teorema "o dobro de um número ímpar é par", subentendemos um quantificador existencial.
4. Para demonstrar o "teorema" "Se São Luís é a capital, então Maranhão é o estado", é suficiente demonstrar que "se Maranhão é o estado, então São Luís é a capital".
5. Provar "A se, e somente se, B" requerer a prova de $A \rightarrow B$ e a prova de $B \rightarrow A$.

Seção 2.2

6. A indução é uma técnica de demonstração apropriada para provar resultados envolvendo todos os números inteiros positivos.
7. A base da indução em uma demonstração indutiva requer que se prove que uma propriedade é válida para $n = 1$.
8. Se a condição para que $P(k + 1)$ seja verdadeira depender da veracidade de valores anteriores a $P(k)$, então devemos usar a indução forte.
9. O caminho para uma demonstração por indução fraca é verificar como o fato de P ser verdadeira no valor $k+1$ depende do fato de P ser verdadeira no valor k .
10. A hipótese indutiva na demonstração por indução para a afirmação

$$1^3 + 2^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

é

$$K^3 = \frac{k^2(k+1)^2}{4}$$

Seção 2.3

11. Uma seqüência definida por

$$S(1) = 7$$

$$S(n) = 3S(n-1) + 2 \quad \text{para } n \geq 2$$

contém o número 215.

12. Algoritmos recursivos são sempre nossas primeiras opções porque são mais eficientes que algoritmos iterativos.
13. Aplicando um algoritmo de busca binária à lista

2, 5, 7, 10, 14, 20

onde $x = 8$ é o item a ser procurado, x nunca será comparado com o valor 5.

14. Uma solução de forma fechada para uma relação de recorrência é obtida pela aplicação de indução matemática à relação de recorrência.
15. $S(n) = 2S(n - 1) + 3S(n - 2) + 5n$ é uma relação de recorrência linear de primeira ordem com coeficientes constantes.

Seção 2.4

16. A análise de um algoritmo geralmente avalia a quantidade de trabalho demandada considerando o pior caso, pois é muito difícil se fazer a avaliação para o caso médio.
17. Algoritmos do tipo dividir para conquistar nos leva a relações de recorrência que não são de primeira ordem.
18. Uma invariante de laço permanece verdadeira até que o loop seja encerrado, ocasião em que ele se torna falso.
19. A correção parcial de um laço em um programa significa que o laço comporta-se corretamente para alguns valores de entrada e não para outros.
20. O algoritmo de busca binária é mais eficiente que o algoritmo de busca sequencial em uma lista ordenada com mais de três elementos.

No Computador

Para os Exercícios 1 a 10, escreva um programa de computador que produza o resultado desejado a partir dos dados de entrada.

1. *Entrada:* Número n de termos em uma progressão geométrica (veja Exercício 17, Seção 2.2), o termo inicial a e a razão r .
Saída: Soma dos primeiros n termos usando:
 - a. iteração
 - b. fórmula do Exercício 17, Seção 2.2
2. *Entrada:* Número n de termos de uma progressão aritmética (veja Exercício 18, Seção 2.2), o termo inicial a e a razão d .
Saída: Soma dos primeiros n termos, usando:
 - a. iteração
 - b. fórmula do Exercício 18, Seção 2.2
3. *Entrada:* Número n
Saída: Soma dos n primeiros cubos, usando:
 - a. iteração, usando apenas multiplicação e adição. Mostre, ao final, o número de multiplicações e adições realizadas
 - b. fórmula do Exercício 8, Seção 2.2, usando apenas multiplicação, adição e divisão. Mostre, ao final, o número de adições, multiplicações e divisões realizadas.
4. *Entrada:* Nenhuma
Saída: Tabela mostrando todo inteiro n , $8 \leq n \leq 100$ como soma de 3s e 5s (veja Exemplo 18).
5. *Entrada:* Cadeia Binária
Saída: Mensagem indicando se a cadeia é um palíndromo (veja Prática 15).
Algoritmo: Use recursão.
6. *Entrada:* Cadeia de caracteres x e um inteiro positivo n .
Saída: Concatenação de n cópias de x .
Algoritmo: Use recursão.
(Algumas linguagens de programação fornecem rotinas internas para manipulação de strings, tais como a concatenação.)
7. *Entrada:* Inteiro positivo n
Saída: n -ésimo valor de seqüência de Fibonacci, usando:
 - a. iteração
 - b. recursão

Introduza um contador em cada versão para indicar o número total de operações de adição utilizadas. Execute cada versão para vários valores de n e, em um gráfico simples, marque o número de adições como função de n para ambas as versões.

8. *Entrada*: Dois inteiros positivos m e n
Saída: $\text{mdc}(m,n)$ **máximo divisor comum** de m e n .
Algoritmo: $\text{mdc}(m,n)$ é definido como o maior inteiro que divide ao mesmo tempo m e n . Suponha que m é menor do que n . Se m divide n então $\text{mdc}(m,n) = m$. Por outro lado, o **algoritmo de Euclides** afirma que o $\text{mdc}(m,n)$ é igual ao maior divisor de m e o resto da divisão de n por m . Por isso, uma relação de recursão é apropriada.
9. *Entrada*: Lista não ordenada de 10 elementos
Saída: Lista ordenada em ordem crescente
Algoritmo: Use o algoritmo de seleção recursiva do Exemplo 28.
10. *Entrada*: Lista ordenada de 10 inteiros e um inteiro x
Saída: Mensagem indicando se x pertence à lista
Algoritmo: Use o algoritmo de busca binária do Exemplo 29.
11. A fórmula $3^n < n!$ é verdadeira para todo $n \geq N$. Escreva um programa para determinar N e depois demonstre o resultado por indução.
12. A fórmula $2^n > n^3$ é verdadeira para todo $n \geq N$. Escreva um programa para determinar N e depois demonstre o resultado por indução.
13. O valor $(1 + \sqrt{5})/2$ conhecido como **razão áurea** é relacionado à sequência de Fibonacci por:

$$\lim_{n \rightarrow \infty} \frac{F(n+1)}{F(n)} = \frac{1 + \sqrt{5}}{2}$$

Verifique este limite calculando $F(n+1)/F(n)$ para $n = 10, 15, 25, 50$ e 100 , e compare este resultado com a razão áurea.

14. Compare o trabalho realizado pelos algoritmos de busca binária e de busca seqüencial em uma lista ordenada de n entradas, pelo cálculo de n e $1 + \log n$ para valores de n de 1 até 100. Apresente os resultados de forma gráfica.